

РО 1.3. Базовое администрирование Linux

Лекция 7. Службы, systemd, журналы и автозагрузка

Цель лекции

- Понять, как Linux запускает и контролирует службы через `systemd`.
- Научиться объяснять состояния служб и действия `start`, `stop`, `restart`, `reload`.
- Научиться проверять автозагрузку и читать журналы `systemd`.
- Сформировать безопасный порядок изменения конфигурации службы.

Список учебных вопросов

1. Служба Linux: назначение и отличие от обычного процесса.
2. systemd как система инициализации и управления службами.
3. Unit-файлы systemd на уровне назначения.
4. Состояния службы: active, inactive, failed.
5. Управление службами: start, stop, restart, reload, status.
6. Автозагрузка служб: enable, disable, is-enabled.
7. Журналы systemd: journalctl.
8. Поиск ошибок службы в журналах.
9. Безопасное изменение конфигурации службы и проверка результата.

1. Служба Linux: назначение

Служба Linux - это программа, которая работает в фоне и предоставляет функцию системе или пользователям.

Примеры служб: SSH для удаленного доступа, cron для планировщика, nginx для веб-сервера.

Служба обычно запускается без прямого участия пользователя и должна переживать выход пользователя из системы.

Администратор отвечает не только за запуск службы, но и за ее состояние, автозагрузку, журналы и безопасность.

Служба и обычная команда

Обычная команда выполняется в терминале и завершается после выполнения задачи.

Служба работает длительно, принимает запросы или выполняет фоновые действия.

У службы есть состояние: запущена, остановлена, завершилась с ошибкой.

Для службы важны зависимости: сеть, файлы конфигурации, права, порты, другие службы.

Служба, процесс и порт

Служба обычно представлена одним или несколькими процессами.

Если служба сетвая, она может слушать TCP или UDP порт.

Например, SSH обычно слушает TCP-порт 22.

Проверка службы не ограничивается status: иногда нужно проверить процесс, порт и доступ с клиента.

Зачем администратору управлять службами

- После установки пакета служба может не запуститься автоматически.
- После изменения конфигурации службу нужно перечитать или перезапустить.
- После перезагрузки сервера нужная служба должна подняться сама.
- Если сервис недоступен, журнал службы помогает найти причину.

2. `systemd`: роль в Linux

`systemd` - это система инициализации и менеджер служб в современных дистрибутивах Linux.

После загрузки ядра `systemd` становится процессом с PID 1.

PID 1 запускает остальные службы и управляет зависимостями.

Для администратора основной инструмент работы с `systemd` - команда `systemctl`.



SYSTEMD КАК ЦЕНТР УПРАВЛЕНИЯ СЛУЖБАМИ

Единая система инициализации и управления системой в современных Linux



systemd — это набор компонентов для управления системой. PID 1 (первый процесс), который запускается ядром и управляет всеми остальными процессами и службами.

1. ЧТО ТАКОЕ SYSTEMD?

systemd — это система инициализации (init system) и менеджер служб для Linux.

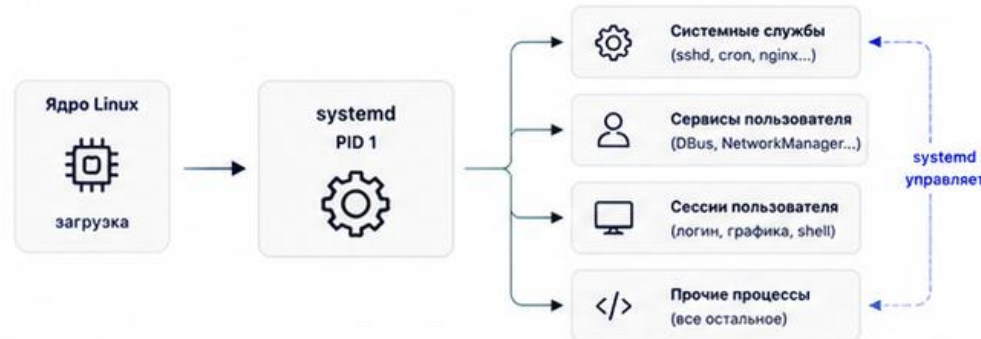
- ✓ Запускает службы при загрузке
- ✓ Управляет процессами и зависимостями
- ✓ Логирование и мониторинг
- ✓ Параллельный запуск для быстрой загрузки
- ✓ Единая точка управления системой



Цель: сделать систему стабильной, управляемой и предсказуемой.

2. КАК РАБОТАЕТ SYSTEMD?

systemd (PID 1) запускается ядром и становится родителем всех процессов.



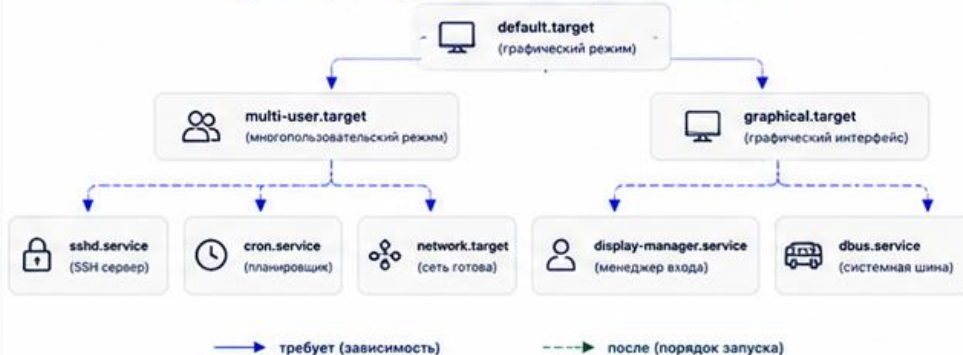
4. УПРАВЛЕНИЕ ЧЕРЕЗ CLI (SYSTEMCTL)

Команда	Описание
systemctl start nginx	запустить службу
systemctl stop nginx	остановить службу
systemctl restart nginx	перезапустить службу
systemctl status nginx	показать статус службы
systemctl enable nginx	включить автозапуск
systemctl disable nginx	выключить автозапуск
systemctl list-units --type=service	список всех служб

```
$ systemctl status sshd
• sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled)
   Active: active (running) since Mon 2024-05-20 10:15:22 +05; 2h 3min ago
     Main PID: 1234 (sshd)
       Tasks: 1 (limit: 4915)
      Memory: 1.3M
     CGroup: /system.slice/ssh.service
             └─1234 /usr/sbin/sshd -o
```

5. ЗАВИСИМОСТИ И ЦЕЛИ (TARGETS)

systemd запускает службы в зависимости от целей и зависимостей.



3. УПРАВЛЕНИЕ СЛУЖБАМИ (UNIT)

systemd управляет системными ресурсами через «unit».

Основные типы unit:

- ⚙️ .service — службы (демоны)
- 🪦 .socket — сокеты
- ▶️ .target — точки синхронизации (цели)
- 🕒 .timer — таймеры (отложенный запуск)
- 📁 .mount — точки монтирования
- 📁 .path — активация по изменению пути
- 👤 .scope — группы процессов



Unit-файлы — это конфигурационные файлы в /etc/systemd/system/ или /usr/lib/systemd/system/

6. ЛОГИ И МОНИТОРИНГ

systemd предоставляет встроенные инструменты для журналов и диагностики.

Команда	Описание
journalctl	показать все логи
journalctl -u nginx	логи конкретной службы
journalctl -b	логи текущей загрузки
journalctl -xe	логи с ошибками (последние)
systemd-analyze	анализ времени загрузки
systemd-analyze blame	что дольше всего запускалось

```
$ journalctl -u nginx -n 5 --no-pager
May 20 10:15:21 server systemd[1]: Starting nginx.service...
May 20 10:15:21 server nginx[1234]: Starting nginx: [ OK ]
May 20 10:15:21 server systemd[1]: Started nginx.service.
```

7. ПРЕИМУЩЕСТВА SYSTEMD

- 🚀 Параллельный запуск ускоряет загрузку
- 🔗 Управление зависимостями надежный порядок запуска
- 📄 Единый подход к службам и логированию
- 🔍 Инструменты диагностики и мониторинга
- ⚙️ Гибкость и расширяемость

8. ГДЕ НАХОДЯТСЯ UNIT-ФАЙЛЫ?

- /etc/systemd/system/ — пользовательские юниты (переопределяют системные)
- /usr/lib/systemd/system/ — системные юниты (из пакетов)
- /run/systemd/system/ — сгенерированные во время выполнения юниты пользователя
- ~/.config/systemd/user/

После изменения unit-файла:

1. systemctl daemon-reload
2. systemctl restart <service>
3. systemctl enable <service>



ВАЖНО



Не редактируйте системные юниты из /usr/lib/systemd/system/ — создавайте переопределения в /etc/systemd/system/



Всегда проверяйте статус: systemctl status <service>



Используйте journalctl для поиска проблем



systemd управляет не только службами, но и всей системой.

systemctl: основной интерфейс администратора

systemctl отправляет команды systemd: показать статус, запустить, остановить, включить автозагрузку.

Пример просмотра версии: **systemctl --version**

Пример списка служб:

systemctl list-units --type=service

Эти команды ничего не меняют, поэтому подходят для первичной ориентации.

Просмотр активных служб

Для просмотра активных service-unit используют:

`systemctl list-units --type=service`

В выводе важны поля UNIT, LOAD, ACTIVE, SUB, DESCRIPTION.

ACTIVE показывает общее состояние, например active или failed.

SUB уточняет состояние, например running или exited.

3. Unit-файл: назначение

Unit-файл - это описание объекта, которым управляет systemd.

Для служб чаще всего используется тип `.service`.

Unit-файл отвечает на вопросы: что запускать, от чего зависит, как включать в автозагрузку.

Администратор должен уметь читать unit-файл, даже если не пишет его с нуля.

Типы unit-файлов

- .service описывает службу.
- .socket описывает socket-активацию.
- .timer описывает запуск по расписанию.
- .mount описывает точку монтирования.
- .target объединяет несколько unit в логическую цель загрузки.

Где лежат unit-файлы

`/lib/systemd/system` или `/usr/lib/systemd/system` содержат unit-файлы из пакетов.

`/etc/systemd/system` содержит локальные изменения администратора.

Файлы в `/etc` имеют приоритет над пакетными файлами.

Правильнее не редактировать пакетный unit напрямую, а использовать `override` или отдельный локальный unit.

Структура service-unit

Секция [Unit] содержит описание и зависимости.

Секция [Service] описывает запуск: ExecStart, пользователь, режим работы.

Секция [Install] описывает привязку к автозагрузке.

Понимание секций помогает читать причину запуска и поведение службы.



СТРУКТУРА SERVICE-UNIT SYSTEMD

Service unit — это конфигурационный файл, который описывает, как systemd должен запускать и управлять службой.



Где находятся unit-файлы?

- Системные: `/etc/systemd/system/` или `/usr/lib/systemd/system/`
- Пользовательские: `~/.config/systemd/user/`
- После изменения файла: `systemctl daemon-reload`

1. ЧТО ТАКОЕ SERVICE-UNIT?

Service unit (служба) описывает длительный процесс в системе: как его запускать, перезапускать, останавливать и в каких условиях.



Преимущества:

- ✓ Автоматический запуск при загрузке
- ✓ Перезапуск при сбоях
- ✓ Управление зависимостями
- ✓ Логирование и отслеживание состояния

2. СТРУКТУРА UNIT-ФАЙЛА

`/etc/systemd/system/myapp.service`

```
[Unit]
# Описание и зависимости юнита
Description=My Application Service
Documentation=https://example.com/docs
After=network.target
Wants=network-online.target
Requires=other-service.service

[Service]
# Настройки запуска и поведения службы
Type=simple
User=myuser
Group=mygroup
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/myapp --config /etc/myapp/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
ExecStop=/bin/kill -TERM $MAINPID
Restart=on-failure
RestartSec=5s
TimeoutStartSec=30s
Environment=ENV="production"
LimitNOFILE=65536

[Install]
# Установка и включение службы
WantedBy=multi-user.target
```

[Unit] — метаданные юнита
Описание и зависимости

[Service] — основной раздел
Команды запуска, окружение, поведение при сбоях

[Install] — установка
Определяет, при каком режиме загрузки юнит будет активен

3. ОСНОВНЫЕ СЕКЦИИ И ПОЛЯ

[Unit] — метаданные и зависимости

Description=	Краткое описание службы
Documentation=	Ссылка на документацию
After=	Запускать после указанных юнитов
Wants=	Необязательная зависимость
Requires=	Обязательная зависимость

[Service] — запуск и управление

Type=	Тип службы (simple, forking, oneshot, notify, exec...)
User= / Group=	Пользователь и группа для запуска
WorkingDirectory=	Рабочая директория
ExecStart=	Команда запуска (обязательно)
ExecReload=	Команда перезагрузки конфигурации
ExecStop=	Команда остановки (по умолчанию SIGTERM)
Restart=	Политика перезапуска (no, on-failure, always...)
RestartSec=	Задержка перед перезапуском
Environment=	Переменные окружения
LimitNOFILE=	Ограничение числа открытых файлов и др.

[Install] — установка

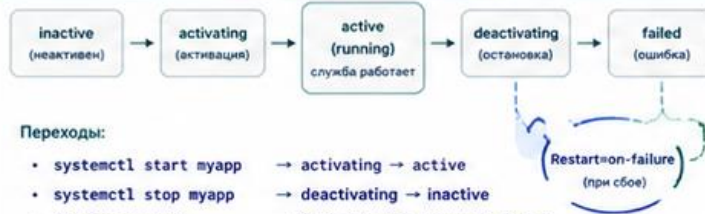
WantedBy=	Включить в указанный target (обычно multi-user.target)
Also=	Дополнительные юниты для установки
RequiredBy=	Юнит, который требует этот юнит

4. ТИПЫ SERVICE (Type=)

Тип	Описание
simple	Запускается основным процессом (по умолчанию)
forking	Процесс форкается в фон (нужен PID-файл)
oneshot	Разовая команда, завершается после выполнения
notify	Приложение само сообщает systemd, что готово (sd_notify)
exec	Похоже на simple, но без оболочки
dbus	Активируется по появлению D-Bus имени

💡 Большинство современных служб используют Type=simple или Type=notify.

5. ЖИЗНЕННЫЙ ЦИКЛ SERVICE



Переходы:

- `systemctl start myapp` → activating → active
- `systemctl stop myapp` → deactivating → inactive
- Ошибка запуска → failed (срабатывает Restart=)

6. ПРИМЕР: ПРОСТОЙ SERVICE

```
# /etc/systemd/system/hello.service
[Unit]
Description=Hello Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/hello.sh
Restart=on-failure
RestartSec=3s

[Install]
WantedBy=multi-user.target
```

Управление:

```
# Перечитать конфигурацию
sudo systemctl daemon-reload
# Запустить службу
sudo systemctl start hello.service
# Проверить статус
sudo systemctl status hello.service
# Включить автозапуск
sudo systemctl enable hello.service
# Остановить службу
sudo systemctl stop hello.service
```

7. ПОЛЕЗНЫЕ КОМАНДЫ



<code>systemctl list-units --type=service</code>	список всех service unit
<code>systemctl status <service></code>	статус службы
<code>journalctl -u <service> -f</code>	логи службы в реальном времени
<code>systemctl is-enabled <service></code>	включена ли служба в автозагрузку
<code>systemctl cat <service></code>	показать содержимое unit-файла

8. ПОЛЕЗНЫЕ TARGETS

multi-user.target	обычный текстовый режим (большинство служб)
graphical.target	графический режим (GUI)
network.target	сеть доступна
network-online.target	сеть полностью готова
rescue.target	режим восстановления
reboot.target / poweroff.target	перезагрузка / выключение



Совет

Используйте `systemctl status <service>` и `journalctl -u <service> -f` для диагностики проблем.



Важно

- После изменения unit-файла: `systemctl daemon-reload`
- Не редактируйте файлы в `/usr/lib/systemd/system/` — перезапишите в `/etc/systemd/system/`
- Используйте `ExecStartPre=` и `ExecStartPost=` для подготовки и действий после запуска.

Просмотр unit-файла без редактирования

Для просмотра unit службы используют: **systemctl cat [имя службы]**

Команда показывает основной unit и возможные override-файлы.

Это безопаснее, чем сразу открывать файл на редактирование.

Если служба называется `ssh.service`, в `systemctl` обычно можно писать `ssh`.

4. Состояния службы

Состояние службы показывает, работает ли она сейчас и были ли ошибки.

active означает, что служба активна.

inactive означает, что служба остановлена.

failed означает, что служба завершилась с ошибкой или не смогла запуститься.

enabled и **disabled** относятся к автозагрузке, а не к текущему запуску.

active, inactive, failed

- active (running): служба работает и обычно имеет активный процесс.
- active (exited): команда запуска завершилась успешно, но постоянного процесса нет.
- inactive: служба сейчас не работает.
- failed: systemd зафиксировал ошибку запуска или выполнения.

Для диагностики failed почти всегда нужен journalctl.

Схема: жизненный цикл службы

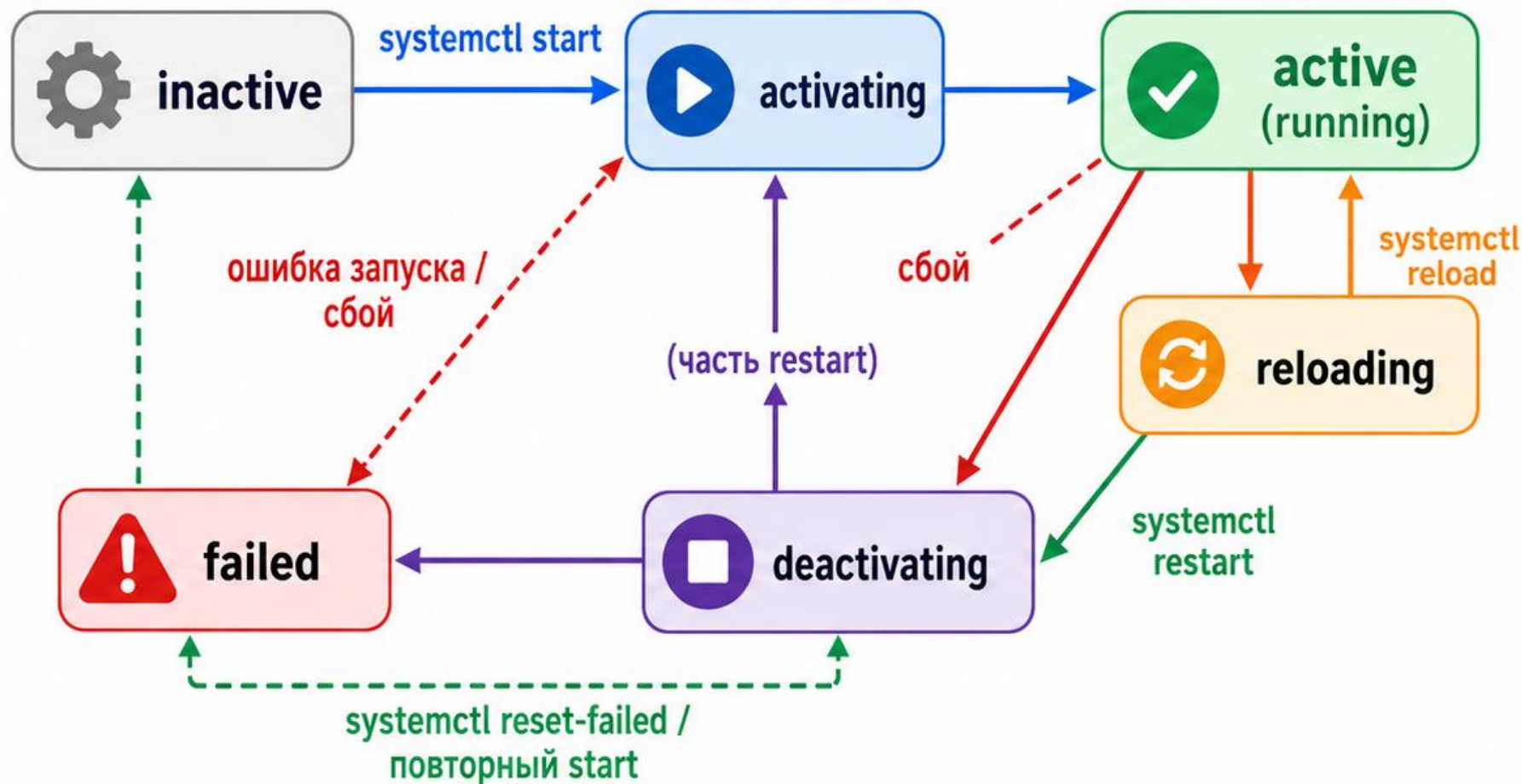
Служба может переходить между состояниями после действий администратора или ошибок.

`start` переводит службу из `inactive` в `active`, если конфигурация корректна.

`stop` переводит `active` в `inactive`.

Ошибка запуска переводит службу в `failed` и требует чтения журнала.

Жизненный цикл службы systemd



>_	Ключевые команды
	<code>systemctl start <service></code>
	<code>systemctl stop <service></code>
	<code>systemctl restart <service></code>
	<code>systemctl reload <service></code>
	<code>systemctl status <service></code>
	<code>journalctl -u <service></code>



Главная идея: systemd переводит службу между состояниями, а администратор управляет этим через `systemctl` и журнал `journalctl`.

Проверка состояния службы

Основная команда проверки:

`systemctl status [имя службы]`

В выводе нужно посмотреть Loaded, Active, Main PID и последние строки журнала.

Loaded показывает, найден ли unit и включена ли автозагрузка.

Active показывает текущее состояние и время последнего изменения.

Как читать `systemctl status`

Если `Active: active (running)`, служба запущена.

Если `Active: failed`, нужно посмотреть код ошибки и журнал.

`Main PID` показывает основной процесс службы.

Последние строки журнала помогают быстро увидеть причину проблемы, но для полной картины лучше `journalctl -u`.

5. Управление службами: start и stop

start запускает остановленную службу сейчас.

Пример запуска SSH: **sudo systemctl start ssh**

stop останавливает службу сейчас.

Пример остановки: **sudo systemctl stop ssh**

После start или stop проверяют результат через
systemctl status ssh

restart: полный перезапуск

restart останавливает службу и запускает ее заново.

Пример: **sudo systemctl restart ssh**

restart применяют после изменений, которые нельзя перечитать мягко.

Минус restart: активные подключения или запросы могут быть прерваны.

На реальном сервере перед restart нужно понимать последствия для пользователей.

reload: перечитать конфигурацию

reload просит службу перечитать конфигурацию без полного перезапуска.

Пример: **sudo systemctl reload nginx**

reload работает только если сама служба поддерживает такую операцию.

Если reload не поддерживается, используют restart.

После reload нужно проверить status и функциональность службы.

status: не действие, а проверка

status не изменяет службу, а показывает ее состояние.

Пример: **systemctl status ssh**

Эту команду выполняют до изменения, чтобы понять исходное состояние.

Ее же выполняют после изменения, чтобы подтвердить результат.

Но status не заменяет клиентскую проверку, если служба предоставляет сетевой сервис.

6. Автозагрузка служб

Автозагрузка отвечает на вопрос: запустится ли служба после перезагрузки сервера.

Запущенная сейчас служба не обязательно включена в автозагрузку.

Включенная в автозагрузку служба не обязательно запущена прямо сейчас.

Поэтому `current state` и `boot state` проверяются разными командами.

enable и disable

enable включает службу в автозагрузку.

Пример: **sudo systemctl enable ssh**

disable убирает службу из автозагрузки.

Пример: **sudo systemctl disable ssh**

После изменения проверяют: **systemctl is-enabled
ssh**

enable --now

Иногда нужно сразу запустить службу и включить ее в автозагрузку.

Для этого используют: **sudo systemctl enable --now ssh**

Команда выполняет две операции: включает автозагрузку и запускает службу.

Проверка после этого двойная: `systemctl is-enabled ssh` и `systemctl status ssh`.

is-enabled и enabled/disabled

is-enabled показывает состояние автозагрузки.

Пример: `systemctl is-enabled ssh`.

enabled означает, что служба должна стартовать при загрузке.

disabled означает, что автоматический старт не настроен.

static означает, что unit не включается напрямую, а запускается как зависимость.

7. Журналы `systemd: journalctl`

`journalctl` читает системный журнал `systemd-journald`. Журнал содержит сообщения ядра, `systemd`, служб и приложений.

Для администратора журнал - основной источник причин ошибки.

Важно читать не только последнюю строку, а последовательность событий.

Схема: путь диагностики через journalctl

Сначала проверяют status, затем переходят к журналу конкретной службы.

После этого ограничивают вывод текущей загрузкой, временем или уровнем важности.

Найденная причина проверяется исправлением и повторным status.

Так диагностика становится последовательной, а не случайной.



ДИАГНОСТИКА СЛУЖБЫ ЧЕРЕЗ JOURNALCTL

journalctl — мощный инструмент для просмотра и анализа журналов systemd и служб.



Все логи systemd и служб собираются в журнал (journald). journalctl позволяет фильтровать, искать и анализировать события.

1. ЧТО ТАКОЕ JOURNALD?

journald — системный демон, который:

- ✓ Собирает логи ядра, systemd и служб
- ✓ Хранит их в бинарном журнале
- ✓ Предоставляет быстрый поиск и фильтрацию
- ✓ Поддерживает структурированные данные



Журналы хранятся в:

/run/log/journal/ (временные)

/var/log/journal/ (постоянные, если включено)

2. БАЗОВЫЕ КОМАНДЫ

Команда	Описание
journalctl	показать все журналы
journalctl -f	следить за логами в реальном времени (как tail -f)
journalctl -b	логи текущей загрузки
journalctl -b -1	логи предыдущей загрузки
journalctl -u <service>	логи конкретной службы
journalctl -u <service> -f	следить за логами службы в реальном времени
journalctl --since "2024-05-20 10:00"	логи с указанного времени
journalctl --until "2024-05-20 12:00"	логи до указанного времени
journalctl --since "1 hour ago"	логи за последний час
journalctl -p err	только ошибки и выше (err, crit, alert, emerg)
journalctl -k	логи ядра (kernel)
journalctl -u <service> --no-pager	без постраничного вывода

3. ПРИМЕРЫ ДЛЯ ДИАГНОСТИКИ СЛУЖБЫ

Задача	Команда	Пример вывода (фрагмент)
Посмотреть логи службы	journalctl -u nginx	May 20 10:15:21 server systemd[1]: Starting nginx... May 20 10:15:21 server nginx[1234]: nginx started
Следить за логами службы	journalctl -u nginx -f	May 20 10:16:10 server nginx[1234]: 200 OK "GET /" May 20 10:16:11 server nginx[1234]: 404 Not Found "GET /x"
Логи за последний час	journalctl -u nginx --since "1 hour ago"	May 20 09:30:00 server nginx[1234]: worker process started May 20 09:45:10 server nginx[1234]: reload configuration
Ошибки службы	journalctl -u nginx -p err	May 20 10:17:05 server nginx[1234]: [error] bind() to 0.0.0.0:80 failed
Предыдущая загрузка	journalctl -b -1 -u nginx	May 20 08:12:01 server systemd[1]: nginx.service: Failed
Контекст вокруг ошибки	journalctl -u nginx -p err -n 20 --no-pager	... 10 lines before ... May 20 10:17:05 server nginx[1234]: [error] 10 lines after ...

4. ФИЛЬТРАЦИЯ И ПОИСК

Опция	Описание
-u <unit>	фильтр по службе (unit)
-p <prio>	фильтр по уровню приоритета (0=emerg ..., 7=debug)
--since, --until	фильтр по времени
--grep <text>	поиск по тексту
-k	только сообщения ядра
_PID=<pid>	фильтр по PID процесса
_COMM=<name>	фильтр по имени процесса



Уровни приоритета:

0 emerg 1 alert 2 crit 3 err
4 warning 5 notice 6 info 7 debug

5. ПОЛЕЗНЫЕ ПРИМЕРЫ КОМАНД

Команда	Что делает
journalctl -u sshd -f	следить за логами SSH
journalctl -u docker.service --since "yesterday"	логи Docker за вчера
journalctl -u myapp --since "2024-05-20 10:00" --until "2024-05-20 11:00"	логи за интервал времени
journalctl -u myapp --grep "error"	поиск строк с "error"
journalctl -u myapp -p warning..alert	только предупреждения и выше
journalctl -u myapp _PID=1234	логи конкретного процесса
journalctl -u myapp -o short-iso	короткий формат даты/времени
journalctl -u myapp -o json-pretty	вывод в формате JSON (удобно для анализа)

6. ФОРМАТЫ ВЫВОДА (-o)

Формат	Описание
-o short	краткий (по умолчанию)
-o short-iso	ISO-формат времени
-o verbose	подробный вывод
-o json	JSON
-o json-pretty	красивый JSON
-o cat	только сообщение (как cat)
-o syslog	в формате syslog

Пример: journalctl -u nginx -o short-iso

```
2024-05-20T10:15:21+03:00 server systemd[1]: Starting nginx...
2024-05-20T10:15:21+03:00 server nginx[1234]: nginx started
```

Пример: journalctl -u nginx -o cat

```
Starting nginx...
nginx started
200 OK "GET /"
404 Not Found "GET /x"
```

7. СОХРАНЕНИЕ И ЭКСПОРТ ЛОГОВ

Команда	Описание
journalctl -u myapp > myapp.log	сохранить вывод в файл
journalctl -u myapp --since "yesterday" > yesterday.log	сохранить логи за период
journalctl -u myapp -o json > myapp.json	экспорт в JSON
journalctl -b -1 -u myapp > boot-previous.log	логи предыдущей загрузки в файл

8. ДИАГНОСТИКА ПРОБЛЕМ: ЧЕК-ЛИСТ

- 1 Служба не запускается → journalctl -u <service> -n 50 --no-pager
- 2 Служба часто падает → journalctl -u <service> --since "1 hour ago" -p err
- 3 Проблемы после перезагрузки → journalctl -b -1 -u <service>
- 4 Поиск ошибок в конфигурации → journalctl -u <service> --grep -i "error"
- 5 Проверка зависимостей → journalctl -u <service> --u systemd -b

9. ПОЛЕЗНЫЕ СОВЕТЫ



Используйте -f для реального времени.
Комбинируйте фильтры (-u, -p, --since, --grep).
Для больших логов используйте -o cat или --no-pager.
JSON-вывод удобен для парсинга скриптами.
Логи хранятся надежно и не теряются при перезагрузке.



ВАЖНО



journalctl читает бинарный журнал, а не текстовые файлы.



Не редактируйте файлы в /var/log/journal вручную!



Используйте systemd-cat для логирования в journald из скриптов:

```
echo "Hello" | systemd-cat -t myscript -p info
```



Проверяйте статус службы вместе с логами:

```
systemctl status <service>
```

Журнал конкретной службы

Для просмотра журнала SSH используют: `journalctl -u ssh --no-pager`.

- u выбирает конкретный unit.

- no-pager выводит текст сразу в терминал без интерактивного просмотрщика.

Такой вывод удобно вставлять в отчет или анализировать через `grep`.

Журнал текущей загрузки

Ключ `-b` ограничивает журнал текущей загрузкой системы.

Пример: `journalctl -u ssh -b --no-pager`.

Это удобно, если старые ошибки уже не относятся к текущей проблеме.

Для анализа серверов после перезагрузки этот фильтр особенно полезен.

Последние строки журнала

Ключ `-n` показывает заданное количество последних строк.

Пример: **`journalctl -u ssh -n 30 --no-pager`**

Ключ `-f` включает наблюдение за журналом в реальном времени.

Пример: **`journalctl -u ssh -f`**

Режим `-f` полезен при проверке подключения клиента или перезапуска службы.

Фильтрация по важности

Ключ -p фильтрует журнал по уровню важности.

Пример ошибок: **journalctl -p err -b --no-pager**

Для службы: **journalctl -u ssh -p warning -b --no-pager**

warning показывает предупреждения и более серьезные события.

err помогает быстро найти критические ошибки, но может скрыть контекст.

8. Поиск ошибок службы

Диагностика начинается с вопроса: служба не запущена, не слушает порт или недоступна клиенту?
systemctl status показывает состояние systemd.
journalctl показывает подробную причину.
ss или клиентская проверка показывают, доступна ли служба по сети.
Нужно идти от факта ошибки к уровню, на котором она возникла.

Алгоритм диагностики failed

Проверить состояние: `systemctl status имя_службы`.

Посмотреть журнал текущей загрузки: `journalctl -u имя_службы -b --no-pager`.

Найти первую содержательную ошибку, а не только последнюю строку.

Проверить конфигурацию службы, если у нее есть команда проверки синтаксиса.

После исправления выполнить `restart` или `reload` и повторить `status`.

Пример диагностики SSH

Сначала смотрят состояние: `systemctl status ssh`.

Затем журнал: `journalctl -u ssh -b --no-pager`

Если служба запущена, проверяют порт: `ss -tlnp | grep ':22'`.

Если порт слушается, проверяют подключение с клиента: `ssh student@адрес_сервера`.

Так отделяют ошибку службы от сетевой ошибки или ошибки учетной записи.

Типовые причины failed

Ошибка синтаксиса в конфигурационном файле.

Занятый порт, который уже слушает другой процесс.

Нет нужного файла, каталога или прав доступа.

Служба ожидает сеть или зависимость, которая не запущена.

После ручной правки unit-файла не выполнен `daemon-reload`.

9. Безопасное изменение конфигурации

- Изменение службы должно быть управляемым: сначала копия, затем правка, затем проверка.
- Перед изменением фиксируют исходное состояние: `systemctl status имя_службы`.
- Конфигурацию меняют небольшими шагами, чтобы легче найти ошибку.
- После правки выполняют проверку синтаксиса, если она доступна.
- Только затем применяют `reload` или `restart`.

Резервная копия конфигурации

- Перед правкой файла делают копию.
- Пример для SSH: `sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak`.
- Копия позволяет быстро вернуть рабочее состояние.
- Имя копии должно показывать, к какому файлу она относится.
- На важном сервере лучше добавлять дату или номер изменения.

Проверка синтаксиса

Некоторые службы умеют проверять конфигурацию до перезапуска.

Для SSH используют: `sudo sshd -t`.

Если команда ничего не вывела, синтаксис обычно корректен.

Для nginx используют: `sudo nginx -t`.

Если проверка показала ошибку, restart выполнять нельзя, пока ошибка не исправлена.

daemon-reload: когда нужен

daemon-reload нужен, если изменен unit-файл systemd или override.

Для перечитывания unit-файлов используют: `sudo systemctl daemon-reload`.

Она не перезапускает службу, а заставляет systemd перечитать описания unit.

После daemon-reload обычно выполняют restart нужной службы.

Если изменен только конфигурационный файл приложения, daemon-reload обычно не нужен.

mask и unmask

mask полностью запрещает запуск службы через systemd.

Это жестче, чем disable: disable убирает автозагрузку, но ручной запуск остается возможным.

Пример запрета: `sudo systemctl mask имя.service`.

Возврат: `sudo systemctl unmask имя.service`.

mask применяют осторожно, когда службу нужно гарантированно заблокировать.

Чек-лист после изменения службы

- Конфигурационный файл сохранен в правильном месте.
- Синтаксис проверен доступной командой.
- `systemctl status` не показывает failed.
- `journalctl` -и не показывает новую критическую ошибку.
- Если служба сетевая, порт и клиентский доступ проверены отдельно.

Типовые ошибки администратора

- Запустить службу через start, но забыть enable для автозагрузки.
- Перепутать restart и reload и прервать активные подключения.
- Смотреть общий журнал вместо журнала конкретной службы.
- Редактировать unit-файл и забыть daemon-reload.
- Считать active достаточной проверкой, не проверив порт или клиентский доступ.

Итоги лекции

- Служба Linux - управляемый фоновый компонент, а не просто процесс.
- `systemd` запускает службы, учитывает зависимости и автозагрузку.
- `systemctl` используется для управления и проверки состояния.
- `journalctl` помогает находить причины ошибок службы.
- Безопасное изменение службы всегда завершается проверкой результата.

Контрольные вопросы

1. Чем служба отличается от обычной команды в терминале?
2. Почему `systemd` имеет PID 1?
3. Чем `start` отличается от `enable`?
4. Когда использовать `reload`, а когда `restart`?
5. Что показывает `systemctl status`?
6. Как посмотреть журнал конкретной службы за текущую загрузку?
7. Когда нужен `systemctl daemon-reload`?
8. Почему `active` не всегда означает, что сервис доступен пользователю?